

SVD-ROM: A Python package for Singular Value Decomposition Reduced Order Modeling of large datasets.

David I. Salvador-Jasin^{1,2}✉, Robert Vava³, Oliver Strickson¹, Louisa van Zeeland¹, Lydia France¹, Peter Yatsyshin¹, and Scott Hosking¹

¹ The Alan Turing Institute, London, United Kingdom ² Cognition, London, United Kingdom ³ Independent Researcher, London, United Kingdom ✉ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) ↗
- [Repository](#) ↗
- [Archive](#) ↗

Editor: [Open Journals](#) ↗

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Despite their high dimensionality, datasets in fluid dynamics, weather, and climate often exhibit low-rank structure, where a small number of dominant patterns explain most of the variability. Singular Value Decomposition (SVD)-based methods, such as the Proper Orthogonal Decomposition (POD) ([Berkooz et al., 1993](#)), provide efficient and interpretable tools for dimensionality reduction in such systems. Dynamic Mode Decomposition (DMD) ([Schmid, 2022](#)) extends this framework to time-resolved data by extracting coherent spatio-temporal structures and their associated dynamics. This enables the construction of low-dimensional, interpretable emulators of complex dynamical systems. However, the adoption of these methods is limited by the practical challenges of working with modern data volumes. We present SVD-ROM, an open-source Python package for SVD-based Reduced Order Modeling (ROM), designed to operate efficiently on large datasets using parallel and out-of-core computation on standard hardware.

Statement of need

Modern datasets in fields such as fluid dynamics, weather, and climate are characterized by high spatial and temporal resolution, typically leading to very large multi-dimensional data arrays that exceed the memory capacity of standard computing resources. Traditional SVD-based algorithms struggle to process such data due to memory constraints and computational bottlenecks. SVD-ROM addresses this challenge by providing a scalable framework that enables efficient computation of SVD-based reduced order models on datasets that would otherwise be intractable. This is achieved through a combination of scalable algorithms and efficient memory management and parallelization. In contrast to other packages, SVD-ROM is designed to operate on standard computing resources, such as laptop computers. It is purely written in Python and does not require specialized configuration or compilation. It is designed to be accessible to researchers and practitioners without requiring deep expertise in high-performance computing, making it a practical choice for ROM of large multi-dimensional arrays.

State of the field

Several open-source packages exist for ROM of dynamical systems, but most are either not designed for large-scale data or require specialized hardware. This section provides a brief overview of some notable packages in the field.

PyLOM ([Eiximeno et al., 2025](#)) is a Python library for ROM in fluid dynamics with algorithms tailored for supercomputers, where parallel operations are implemented using Message Passing Interface (MPI). PySPOD ([Mengaldo & Maulik, 2021](#)) is a package for Spectral Proper

Orthogonal Decomposition (SPOD) (Schmidt & Colonius, 2020), a frequency-domain extension of POD for statistically stationary time-resolved data, closely related to DMD (Towne et al., 2018). Similarly to PyLDM, PySPOD is designed for high-performance computing environments using MPI for parallelization. PyDMD (Ichinaga et al., 2024) is a Python package that implements DMD and several of its major variants. While it includes a number of cutting-edge methods developed to handle real-world data, it is not specifically designed to handle datasets larger than memory. SVD-ROM uses PyDMD internally to perform DMD at scale. pyMOR (Milk et al., 2016) is a library of ROM techniques for reducing the computational complexity of solving parametrized partial differential equation problems. While SVD-ROM is a data-centric/scalable ROM library, pyMOR is a model-centric ROM framework. Finally, EZyRB (Demo et al., 2018) is a general-purpose Python ROM framework focused on POD/reduced-basis methods and parametric model reduction, including interpolation of reduced-order solutions. While EZyRB provides a broader ROM toolbox than SVD-ROM, it is not specifically optimized for large-scale data processing.

Mathematics

Spatio-temporal data is arranged into an $(m \times n)$ snapshot matrix $\mathbf{X}$, whose rows index spatial locations and whose columns index time, with $m \gg n$ in the extremely tall-and-skinny case typical of fluid dynamics and $m > n$ by a more moderate margin in weather and climate applications. Its SVD is $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$, where $\mathbf{U}$ and $\mathbf{V}$ hold the left and right singular vectors and $\mathbf{\Sigma}$ is the rectangular diagonal matrix whose entries are the singular values σ_j , ordered by decreasing magnitude. ROM proceeds by retaining only the leading k singular values, giving the rank k approximation $\mathbf{X}_k = \mathbf{U}_k\mathbf{\Sigma}_k\mathbf{V}_k^*$, which captures most of the variance of $\mathbf{X}$ when $k \ll n$.

POD is the direct application of the truncated SVD to the (optionally mean-removed) fluctuating field $\mathbf{X}'$, scaled by $1/\sqrt{n}$ so that modal energies do not depend on the number of snapshots:

$$\mathbf{X}'(\mathbf{x}, t) \approx \sum_{j=1}^k \phi_j(\mathbf{x}) a_j(t),$$

where the orthonormal spatial modes ϕ_j are the left singular vectors $\mathbf{U}_k$, the modal energies are $\lambda_j = \sigma_j^2$, and the time coefficients $a_j(t)$ follow from $\mathbf{\Sigma}_k\mathbf{V}_k^*$. Extended POD (Borée, 2003) correlates these modes with a second field $\mathbf{C}$ measured simultaneously in time, by projecting its fluctuating part onto the time coefficients as $\chi_j = (\lambda_j n)^{-1} \sum_i a_{ij} c'_i$, whose norm measures the energy in $\mathbf{C}'$ that is linearly correlated with mode j .

DMD (Schmid, 2022) complements this purely spatial reduction by also extracting temporal dynamics, seeking a decomposition $\mathbf{X} \approx \mathbf{\Phi}\mathbf{B}\mathbf{T}(\omega)$ into DMD modes $\mathbf{\Phi}$, amplitudes $\mathbf{B}$ and temporal dynamics $\mathbf{T}(\omega)$, whose j -th row is of the form $e^{\omega_j t}$ with ω_j the complex frequency (oscillation rate and growth or decay) governing the evolution of the j -th mode. Because these dynamics are continuous in time, they can be extrapolated beyond the training window to produce forecasts. SVD-ROM uses Optimized DMD (Askham & Kutz, 2018), which solves the exponential fitting problem directly by variable projection and is therefore robust to noise and to unevenly sampled snapshots. Crucially, the fit is performed in the SVD latent space,

$$\tilde{\mathbf{\Phi}}\tilde{\mathbf{B}}, \tilde{\omega} = \arg \min_{\tilde{\mathbf{\Phi}}\tilde{\mathbf{B}}, \tilde{\omega}} \|\mathbf{\Sigma}_k\mathbf{V}_k^* - \tilde{\mathbf{\Phi}}\tilde{\mathbf{B}}\mathbf{T}(\tilde{\omega})\|_F,$$

with the resulting $(k \times k)$ latent modes projected back to physical space via $\mathbf{\Phi} = \mathbf{U}_k\tilde{\mathbf{\Phi}}$. This encoder (truncated SVD) $\rightarrow$ processor (Optimized DMD) $\rightarrow$ decoder (orthogonal projection) framework is what allows SVD-ROM to fit DMD models, and their bagged counterparts with uncertainty quantification (Sashidhar & Kutz, 2022), on snapshot matrices that are far too large to be handled directly.

84 Software design

85 SVD-ROM is built on top of Xarray ([Hoyer & Hamman, 2017](#)) and Dask ([Rocklin, 2015](#)),
 86 and operates throughout on chunked, Dask-backed Xarray DataArray objects. This choice
 87 underpins both of the package's design goals. Firstly, labelled dimensions and coordinates mean
 88 that users work with named physical axes (e.g. time, latitude, longitude, etc.) rather than
 89 raw matrix indices, and are preserved across decomposition, reconstruction and forecasting.
 90 Secondly, the Dask backend provides the lazy, out-of-core and parallel execution that allows
 91 datasets larger than memory to be processed on a single machine, streaming from and to
 92 on-disk Zarr stores or NetCDF files.

93 All models share a common interface through an abstract `DecompositionModel` base class
 94 exposing the `fit` and `reconstruct` methods. The computational core is the `TruncatedSVD` class,
 95 which offers a choice between a deterministic, communication-efficient QR factorization for
 96 tall-and-skinny matrices ([Benson et al., 2013](#)), and a randomized algorithm ([Halko et al., 2011](#)),
 97 trading accuracy for speed and relaxed chunking requirements on very large arrays. POD extends
 98 `TruncatedSVD` with the appropriate energy normalization, mean removal and extended-POD
 99 functionality. `OptDMD` consumes the pre-computed SVD factors directly and solves the resulting
 100 low-dimensional nonlinear least-squares problem via the variable-projection Optimized DMD
 101 algorithm of the `PyDMD` package, with optional parallelized bootstrap aggregation for uncertainty
 102 quantification ([Sashidhar & Kutz, 2022](#)) and Hankel time-delay embedding ([Brunton et al., 2017](#))
 103 for systems with insufficient rank. Because the singular triplet is computed once and
 104 reused, a single decomposition can feed several downstream Optimized DMD models at no
 105 additional cost.

106 Throughout, users retain explicit control over what is materialized in memory: `compute_*`
 107 flags and companion methods make it possible to build a Dask computational graph and defer
 108 its evaluation, so that expensive quantities such as spatial modes or time coefficients are
 109 only computed when needed. The package is written in pure Python with no compilation or
 110 MPI configuration required, is fully type annotated, and ships optional extras for weather and
 111 climate workflows (`weather_utils`). An `SPOD` class is currently a placeholder, reflecting the
 112 intended extension of the same interface to further SVD-based methods in future releases.

113 Example

114 SVD-ROM fits a DMD model to a 36 GB slice of the ERA5 reanalysis dataset ([Hersbach et al., 2020](#))
 115 and forecasts 45 days beyond the training window in approximately 10 minutes on a
 116 10-core, 32 GB MacBook M1 Pro. The slice holds a single variable (temperature at the 500
 117 hPa pressure level) on a (time, latitude, longitude) grid: 8,766 snapshots at 4-hourly
 118 resolution between 1 Jan 2016 and 31 Dec 2019, on a 0.25° grid of 721 × 1,440 points, giving
 119 a (1,038,240 × 8,766) snapshot matrix.

```
import xarray as xr

from svdrom import OptDMD, TruncatedSVD
from svdrom.preprocessing import StandardScaler, variable_spatial_stack

# Open the store lazily, remove the time mean, and stack latitude and longitude
# into the "samples" dimension of a tall-and-skinny snapshot matrix.
X = xr.open_dataarray("era5_slice.zarr", chunks="auto")
scaler = StandardScaler()
X = variable_spatial_stack(scaler(X), dims=("latitude", "longitude")).T

# Encode with a randomized truncated SVD, then fit Optimized DMD on the triplet.
svd = TruncatedSVD(
    n_components=40, algorithm="randomized", rechunk=True, compute_var_ratio=True
).fit(X, n_power_iter=2, n_oversamples=15)
```

```
dmd = OptDMD(n_modes=40, time_units="h").fit(svd.u, svd.s, svd.v)
```

```
# Forecast, keep the real part, restore the grid and the mean, and stream to disk.
forecast = dmd.forecast(forecast_span="45 D").real.unstack() + scaler.mean
forecast.to_zarr("forecast_45_days.zarr", mode="w")
```

Three SVD-ROM calls carry the whole workflow, and the data never leaves the labelled, chunked representation. Only the two fit calls and the mean removal are eager; everything else builds a Dask graph, so the forecast is returned on the original latitude/longitude grid with its time coordinate extended past the training window, and streams to Zarr chunk by chunk without ever being held in memory whole. Of the ten minutes, the randomized SVD accounts for roughly seven; mean removal, the DMD fit and the forecast take about one minute each. The 40 retained components explain over 80% of the variance of X , and the DMD fit operates on the resulting $(40 \times 8,766)$ latent triplet rather than on the full matrix, which is why it runs in-memory with a NumPy backend. Because that triplet is computed once and reused, further DMD models — at different ranks, or bagged for uncertainty quantification — have a small additional computational cost. The `demos/` directory contains extended versions of this workflow, including retrieval of the ERA5 slice and validation of the forecast against climatology.

Research impact statement

SVD-ROM was developed as part of the Environmental Forecasting mission at The Alan Turing Institute in London, with the aim of making a computationally efficient and interpretable reduced order modeling and forecasting tool accessible to researchers and practitioners in fluid dynamics, weather, and climate. A major motivation was to enable the analysis of large datasets that are otherwise intractable on standard computing resources. SVD-ROM and related work has been presented at several conferences and workshops, including Climate Informatics 2026 in Lausanne (Salvador-Jasin et al., 2026), a PyData Meetup in London in February 2026 (Salvador-Jasin & Vava, 2026), and FOSDEM 2025 in Brussels (Salvador Jasin, 2025). The package is under active development and used by researchers at The Alan Turing Institute to explore use cases in climate and weather forecasting.

AI usage disclosure

Generative AI tools were used to assist with code development, to edit the documentation, and to review this manuscript. All AI-assisted output was reviewed, tested, and approved by the authors, who take full responsibility for the content of the software and this paper.

Acknowledgements

We acknowledge contributions from J. Nathan Kutz (Autodesk Research, UK) and Benet Eiximeno Franch (NVIDIA, Spain) for their advice on the design of a scalable DMD implementation, and from the PyDMD development team for their support in integrating Optimized DMD into SVD-ROM.

References

- Askham, T., & Kutz, J. N. (2018). Variable projection methods for an optimized dynamic mode decomposition. *SIAM Journal on Applied Dynamical Systems*, 17, 380–416. <https://doi.org/10.1137/M1124176>
- Benson, A. R., Gleich, D. F., & Demmel, J. (2013). Direct QR factorizations for tall-and-skinny matrices in MapReduce architectures. *2013 IEEE International Conference on Big Data*, 264–272. <https://doi.org/10.1109/BigData.2013.6691583>

- 160 Berkooz, G., Holmes, P., & Lumley, J. L. (1993). The proper orthogonal decomposition in
161 the analysis of turbulent flows [Journal Article]. *Annual Review of Fluid Mechanics*, 25,
162 539–575. <https://doi.org/10.1146/annurev.fl.25.010193.002543>
- 163 Borée, J. (2003). Extended proper orthogonal decomposition: A tool to analyse correlated
164 events in turbulent flows. *Experiments in Fluids*, 35, 188–192. <https://doi.org/10.1007/s00348-003-0656-3>
- 166 Brunton, S. L., Brunton, B. W., Proctor, J. L., Kaiser, E., & Kutz, J. N. (2017). Chaos
167 as an intermittently forced linear system. *Nature Communications*, 8(1), 19. <https://doi.org/10.1038/s41467-017-00030-8>
- 168 Demo, N., Tezzele, M., & Rozza, G. (2018). EZyRB: Easy Reduced Basis method. *The
169 Journal of Open Source Software*, 3(24), 661. <https://doi.org/10.21105/joss.00661>
- 171 Eiximeno, B., Miró, A., Begiashvili, B., Valero, E., Rodriguez, I., & Lehmkuhl, O. (2025).
172 PyLOM: A HPC open source reduced order model suite for fluid dynamics applications.
173 *Computer Physics Communications*, 308, 109459. <https://doi.org/10.1016/j.cpc.2024.109459>
- 174 Halko, N., Martinsson, P. G., & Tropp, J. A. (2011). Finding structure with randomness:
175 Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*,
176 53(2), 217–288. <https://doi.org/10.1137/090771806>
- 178 Hersbach, H., Bell, B., Berrisford, P., Hirahara, S., Horányi, A., Muñoz-Sabater, J., Nicolas,
179 J., Peubey, C., Radu, R., Schepers, D., Simmons, A., Soci, C., Abdalla, S., Abellan,
180 X., Balsamo, G., Bechtold, P., Biavati, G., Bidlot, J., Bonavita, M., ... Thépaut, J.-N.
181 (2020). The ERA5 global reanalysis. *Quarterly Journal of the Royal Meteorological Society*,
182 146(730), 1999–2049. <https://doi.org/10.1002/qj.3803>
- 183 Hoyer, S., & Hamman, J. (2017). Xarray: N-d labeled arrays and datasets in python. *Journal
184 of Open Research Software*, 5(1), 10. <https://doi.org/10.5334/jors.148>
- 185 Ichinaga, S. M., Andreuzzi, F., Demo, N., Tezzele, M., Lapo, K., Rozza, G., Brunton, S. L., &
186 Kutz, J. N. (2024). PyDMD: A python package for robust dynamic mode decomposition.
187 *Journal of Machine Learning Research*, 25(417), 1–9. <http://jmlr.org/papers/v25/24-0739.html>
- 188 Mengaldo, G., & Maulik, R. (2021). PySPOD: A Python package for spectral proper orthogonal
189 decomposition (SPOD). *Journal of Open Source Software*, 6(60), 2862. <https://doi.org/10.21105/joss.02862>
- 190 Milk, R., Rave, S., & Schindler, F. (2016). pyMOR – generic algorithms and interfaces
191 for model order reduction. *SIAM Journal on Scientific Computing*, 38(5), S194–S216.
192 <https://doi.org/10.1137/15M1026614>
- 193 Rocklin, M. (2015). Dask: Parallel computation with blocked algorithms and task scheduling.
194 *Proceedings of the 14th Python in Science Conference*, 126–132. <https://doi.org/10.25080/majora-7b98e3ed-013>
- 195 Salvador Jasin, D. (2025). *Explainable forecasting from big weather data: Rapid and sustainable
196 solutions*. Zenodo. <https://doi.org/10.5281/zenodo.14887704>
- 197 Salvador-Jasin, D., & Vava, R. (2026). *SVD-ROM: Reduced order modeling of huge arrays using
198 the singular value decomposition*. Zenodo. <https://doi.org/10.5281/zenodo.18468075>
- 199 Salvador-Jasin, D., Vava, R., Strickson, O., Zeeland, L. van, France, L., Eiximeno Franch,
200 B., Yatsyshin, P., Hosking, S., & Kutz, J. N. (2026). *SVD-ROM: Scalable reduced order
201 modeling of weather and climate data using the singular value decomposition*. Zenodo.
202 <https://doi.org/10.5281/zenodo.19710711>
- 203 Sashidhar, D., & Kutz, J. N. (2022). Bagging, optimized dynamic mode decomposition for

- 207 robust, stable forecasting with spatial and temporal uncertainty quantification. *Philosophical*
208 *Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*,
209 *380*(2229), 20210199. <https://doi.org/10.1098/rsta.2021.0199>
- 210 Schmid, P. J. (2022). Dynamic Mode Decomposition and Its Variants. *Annual Review of Fluid*
211 *Mechanics*, *54*, 225–254. <https://doi.org/10.1146/annurev-fluid-030121-015835>
- 212 Schmidt, O. T., & Colonius, T. (2020). Guide to spectral proper orthogonal decomposition.
213 *AIAA Journal*, *58*(3), 1023–1033. <https://doi.org/10.2514/1.J058809>
- 214 Towne, A., Schmidt, O. T., & Colonius, T. (2018). Spectral proper orthogonal decomposition
215 and its relationship to dynamic mode decomposition and resolvent analysis. *Journal of*
216 *Fluid Mechanics*, *847*, 821–867. <https://doi.org/10.1017/jfm.2018.283>

DRAFT